

ECON1059 Seminar 3

Notes for the Seminar Series on Python Programming for Economics (ECON1059, Fall 2025)

Juncheng ZHANG, PhD Candidate in Economics, UNNC.

Research interest: Behavioral and Experimental Economics.

Email: Juncheng.ZHANG@nottingham.edu.cn

Lists

- A **list** is an **ordered** and **mutable** collection in Python.
- It can store **different data types** (e.g., integers, strings, floats), with elements separated by commas.

Common List Operations & Methods

Method/Operations	Description
<code>list.append(x)</code>	Add <code>x</code> to the end of the list
<code>list.extend(iterable)</code>	Add all elements from another iterable to the end
<code>list.pop(index)</code>	Remove and return the element at <code>index</code> (default: last item)
<code>list.remove(x)</code>	Remove the first occurrence of value <code>x</code>
<code>list.index(x)</code>	Return the first index where <code>x</code> appears
<code>list.count(x)</code>	Return how many times <code>x</code> appears in the list
<code>list[a:b]</code>	Slice — get elements from index <code>a</code> to <code>b-1</code> (left-closed, right-open)
<code>len(list)</code>	Returns the number of elements in the list.

```
In [1]: ## Example 1 Create a list
my_list = [1, 2, 3, 4]
```

```
In [2]: ## Example 2 Access elements
print(my_list[0]) ### output: 1
print(my_list[3]) ### output: 4
print(my_list[-2]) ### output: 3
```

1
4
3

```
In [3]: ## Example 3 Modify elements
my_list[1] = 5
print(my_list)
```

[1, 5, 3, 4]

```
In [4]: ## Example 4 Append elements
my_list.append(6)
print(my_list)
```

[1, 5, 3, 4, 6]

```
In [5]: ## Example 5 List Extend(two methods)
new_list = [7, 8, 9]
my_list.extend(new_list)
print(my_list)    ### outputs: [1, 5, 3, 4, 6, 7, 8, 9]

new_list2 = [7, 8, 9]
new_list3 = my_list + new_list2
print(new_list3)  ### outputs: [1, 5, 3, 4, 6, 7, 8, 9, 7, 8, 9]
```

[1, 5, 3, 4, 6, 7, 8, 9]

[1, 5, 3, 4, 6, 7, 8, 9, 7, 8, 9]

```
In [6]: ## Example 6 Delete Element(two methods)
my_list.pop(3)
print(my_list)    ### outputs: [1, 5, 3, 6, 7, 8, 9]

new_list4 = [1, 2, 3, 4]
del new_list4[3]
print(new_list4)  ### outputs: [1, 2, 3]
```

[1, 5, 3, 6, 7, 8, 9]

[1, 2, 3]

```
In [7]: ## Example 7 Remove a Value (first)
my_list.remove(5)
print(my_list)
```

[1, 3, 6, 7, 8, 9]

```
In [8]: ## Example 8 Find index (first index)
print(my_list.index(7))
```

3

```
In [9]: ## Example 9 Count number
print(my_list.count(7))
```

1

```
In [10]: ## Example 10 Slice a list
# gets elements at index 1 and 2
new_list5 = my_list[1:3]
print(new_list5)
```

[3, 6]

```
In [11]: ## Example 11 Length of a list
print(len(my_list))
print(len(new_list5))
```

6

2

range() Function (Quick check)

The `range()` function in Python is used to **generate a sequence of numbers** — often for use in loops, especially `for` loops.

`range(start, stop, step)`

Parameter	Description	Default
<code>start</code>	First number in the sequence (included)	<code>0</code>
<code>stop</code>	Number where the sequence ends (not included)	<code>—</code>
<code>step</code>	Difference between each number (increment)	<code>1</code>

- Create a list (using range) with items
 - 0, 1, 2, ..., 8
 - 5, 6, 7
 - 2, 4, 6, 8, 10
 - -3, -6, -9

```
In [12]: list_1 = list(range(9))
print(f"list_1 = {list_1}\n")

list_1 = [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

```
In [13]: list_2 = list(range(5, 8))
print(f"list_2 = {list_2}\n")

list_2 = [5, 6, 7]
```

```
In [14]: list_31 = list(range(2, 12, 2))
list_32 = list(range(2, 11, 2))
print(f"Both versions of list_3 yield\n {list_31}\n {list_32}\n")

Both versions of list_3 yield
[2, 4, 6, 8, 10]
[2, 4, 6, 8, 10]
```

```
In [15]: list_4 = list(range(-3, -12, -3))
print(f"list_4 = {list_4}\n")

list_4 = [-3, -6, -9]
```

- Create a list of lists, where the second dimension lists have the following values [1,2,3], [4,5,6], and [7,8,9]

```
In [16]: # Answer 1. Manually
list_51 = [[1,2,3], [4,5,6], [7,8,9]]
print(list_51)

[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
In [17]: # Answer 2. range()
list_52 = [list(range(1, 4)), list(range(4, 7)), list(range(7, 10))]
print(list_52)
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
In [18]: # Answer 3. with a for loop
list_53 = []
for x_min in range(3):
    list_53.append(list(range(1 + 3*x_min, 4 + 3*x_min)))
print(list_53)
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

- Using the above lists, try to access a particular entry.
- Experiment with the other list methods such as append(), pop(), etc.

Left as an exercise for you to play with.

```
In [19]: # List (list of lists)
list_54 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
In [20]: # Access elements
print(list_54[0])          # → [1, 2, 3]   (first row)
print(list_54[0][1])       # → 2          (row 0, column 1)
print(list_54[-1][-1])     # → 9          (last row, last element)
```

```
[1, 2, 3]
2
9
```

```
In [21]: # Append a new row
list_54.append([10, 11, 12])
print(list_54)
# → [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
```

```
In [22]: # Append to a sublist (add element to first row)
list_54[0].append(12)
print(list_54)
# → [[1, 2, 3, 12], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
```

```
[[1, 2, 3, 12], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
```

```
In [23]: # Pop from a sublist
removed = list_54[0].pop(3)
print(removed)          # → 12
print(list_54)
# → [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
```

```
12
[[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
```

Drawing Time

- Create a function that prints a
 - square of dimensions 3x3
 - a triangle (you can choose the particular shape)
 - a christmas tree

► Hints

- Once you successfully drawn the above, go back to your code and see if you can turn these into a general function where the dimension of these objects are flexible (hint: loops are your friend now!)

```
In [24]: print("###")
print("###")
print("###")
```

```
###
###
###
```

```
In [25]: print("#")
print("##")
print("###")
```

```
#
##
###
```

```
In [26]: print("  *")
print("  **")
print("  ****")
print("  *****")
print("  *****")
print("  *")
```

```
  *
  **
  ****
  *****
  *****
  *
```

```
In [27]: def print_square(dim, char):
    for row in range(dim):
        row_to_print = ""
        for column in range(dim):
            row_to_print += char
        print(row_to_print)
```

```
print_square(4, "@")
```

```
@@@@
@@@@
@@@@
@@@@
```

- I am allowing for the character used for drawing the square to be flexible. But you can fix it too.
- Also, I am using a double loop. Using `print(n*char)` we have seen before works too

```
In [28]: # Actually, you may want to generalize the square function to a
def print_rectangular(dim_x, dim_y, char):
```

```

    # To code as practice.
    pass

# And then:
def print_square(dim, char):
    print_rectangular(dim, dim, char) # Beautiful!

```

- To write an empty function, use the keyword `pass`

```

In [29]: # Draw a rectangular shape
def print_rectangular(dim_x, dim_y, char):
    for row in range(dim_y):
        print(char * dim_x)

print_rectangular(2, 4, "@")

```

```

@@
@@
@@
@@

```

```

In [30]: # Draw a square by reusing the rectangular function
def print_square(dim, char):
    print_rectangular(dim, dim, char)

print_square(4, "@")

```

```

@@@@
@@@@
@@@@
@@@@

```

```

In [31]: def print_lazy_triangle(dim, char):
    for row in range(dim):
        row_to_print = ""

        # What happens if we omit the "1 + "?
        for column in range(1 + row):
            row_to_print += char
        print(row_to_print)

    print_lazy_triangle(4, "#")

```

```

#
##
###
####

```

```

In [32]: def print_lazy_triangle(dim, char):
    # Outer loop: controls the number of rows to print
    for row in range(dim):

        # Start each row as an empty string
        # We will "grow" this string by adding characters in the inner loop
        row_to_print = ""

        # Inner loop: controls how many characters we add to the current row
        # Note: using (1 + row) ensures the triangle grows as 1, 2, 3, ...

```

```

# If we omit the "1 +", the first row would be empty (0 characters)
for column in range(1 + row):
    # Add one character at a time
    row_to_print += char

# After finishing the inner loop, print the entire row
print(row_to_print)

print_lazy_triangle(4, "#")

# Out:
# #   (Row 0, range(1) -> contains 1 character "#")
# ##  (Row 1, range(2) -> contains 2 characters "##")
# ### (Row 2, range(3) -> contains 3 characters "###")
# #### (Row 3, range(4) -> contains 4 characters "####")
#
##
###
####

```

```

In [33]: def print_christmas_tree(height, char):
# Note: Height determines rows of needles.
# Height must be 2 or larger.

if height > 1:
    # Build the 'needle part'
    for row in range(height):
        background_left = ""
        foreground = ""

        # The easiest way to create the 'text' is multiplication.
        # For loops work, but I find them a bit more complicated.
        background_left = (height - row - 1)*" " # Empty space.
        foreground = (1 + 2*row)*char
        print(background_left + foreground)

    # Add stem
    stem = (height - 1)*" " + char
    print(stem)
else:
    print("Error, a Christmas tree must have at least a height of 2")

print_christmas_tree(5, "*")

```

```

*
***
*****
*****
*****
*

```

Christmas Tree — Row Breakdown (Example: height = 5, char = "*")

Row	background_left (spaces)	foreground (stars)	Printed Result
0	(5 - 0 - 1) = 4 spaces	1 + 2*0 = 1 star	" *"

Row	background_left (spaces)	foreground (stars)	Printed Result
1	$(5 - 1 - 1) = 3$ spaces	$1 + 2*1 = 3$ stars	" ***"
2	$(5 - 2 - 1) = 2$ spaces	$1 + 2*2 = 5$ stars	" *****"
3	$(5 - 3 - 1) = 1$ space	$1 + 2*3 = 7$ stars	" *****"
4	$(5 - 4 - 1) = 0$ spaces	$1 + 2*4 = 9$ stars	"*****"
Stem	$(5 - 1) = 4$ spaces	1 star	" *"

Explanation:

- `background_left` controls **how many leading spaces** we print to center the tree.
- `foreground` controls **how many characters (`char`)** appear in that row.
- Each row increases by **2 characters**, forming a symmetric triangle.

for vs. while loops

- Use a for-loop to loop over each person in the dataset below and print their name, age, and hobby (maybe nicely formatted using f-strings) in the dataset

```
dataset = [['Marcus', 34, 'Running'], ['Lars', 37, 'Play Guitar'], ['Johannes', 45, 'Cycling']]
```

- Next, do the same using a while loop. (hint: use `len(dataset)` to get the number of entries in the (highest level) list)

```
In [34]: dataset = [['Marcus', 34, 'Running'],
                  ['Lars', 37, 'Play Guitar'],
                  ['Johannes', 45, 'Cycling']]

for datapoint in dataset:
    print(f"Name: {datapoint[0]}, Age: {datapoint[1]}, Hobby: {datapoint[2]}")
```

```
Name: Marcus, Age: 34, Hobby: Running
Name: Lars, Age: 37, Hobby: Play Guitar
Name: Johannes, Age: 45, Hobby: Cycling
```

```
In [35]: dataset = [['Marcus', 34, 'Running'],
                  ['Lars', 37, 'Play Guitar'],
                  ['Johannes', 45, 'Cycling']]

i = 0
while i < len(dataset):
    print(f"Name: {dataset[i][0]}, Age: {dataset[i][1]}, Hobby: {dataset[i][2]}")
    i += 1
```

```
Name: Marcus, Age: 34, Hobby: Running
Name: Lars, Age: 37, Hobby: Play Guitar
Name: Johannes, Age: 45, Hobby: Cycling
```

Editing Lists

- Suppose you have a list of simple numbers collected from answers to survey questions. For example, you may have asked people "how many job applications do you plan to make daily". It turns out that some people actually claimed crazy plans such as 10 000 or 500. (that did not match their subsequent real life choices)

- Using a simple example list that you create, write an algorithm that creates a copy of this list, but where extreme values are replaced with something sensible like 20.
- Afterwards, turn this algorithm into a general function

Hints and further comments for this question

- don't manipulate the original list (bad practise). Instead store values to a new list
- For real data analysis, it is usually better to treat items as 'missing' (and omit them from analysis) instead of arbitrarily changing them. One way to do that would be to set it to -1, if all other numbers are positive, and omit negative values from the analysis

```
In [36]: def censor_values(list_values, max_value_allowed):  
# Replaces any item in list_values that  
# exceeds the maximum value allowed by the  
# maximum value allowed and returns the new list.  
  
new_list = []  
for value in list_values:  
    if value <= max_value_allowed:  
        new_list.append(value)  
    else:  
        new_list.append(max_value_allowed)  
  
return new_list  
  
my_list = [1, 100, 10000, 500, 4]  
print(censor_values(my_list, 200))
```

```
[1, 100, 200, 200, 4]
```